

← Go Back

Hardware

GIGA R1 WiFi



Home / Hardware / GIGA R1 WiFi /  
**Arduino GIGA R1 User Manual**

# Arduino GIGA R1 User Manual

Learn how to set up the GIGA R1, get a quick overview of the components, information regarding pins and how to use different Serial (SPI, I2C, UART), and much, much more.

Author · Jacob Hylén · Last revision · 12/13/2024

The **Arduino GIGA R1** is one of our most feature-packed Arduino boards to date, supported by the same powerful, ML-capable, dual-core microcontroller found in the Pro family's Portenta H7. It features support for display connectors, USB-host, an Audio Jack, an Arducam connector, a CAN bus, 4 UART Serial Ports, 2 I2C buses, dedicated DAC Pins, and much, much more.

This article is a collection of resources and guides to make use of every great feature of this powerful hardware.

You can also visit the documentation platform for the **GIGA R1**.

## Datasheet

The full datasheet is available as a downloadable PDF from the link below:

[Download the GIGA R1](#)

ON THIS PAGE

Datasheet

- Arduino Cloud
- Power Supply
- Installation
- Board Package +
- Boot0
- STM32H747XI Microcontroller
- Memory +
- Radio Module +
- Ethernet
- Audio Jack +
- MIPI DSI®
- USB Features +
- RTC +
- Camera Interface
- JTAG
- CAN Bus
- SPI
- I2C Pins
- Serial/UART Pins
- Pins +
- Interrupts

## Arduino Cloud

The GIGA R1 WiFi is compatible with the [Arduino Cloud](#), a Cloud service that allows you to create IoT applications in just minutes.



Visit the [Getting Started with Arduino Cloud](#) guide for more information.

## Power Supply

To power the **GIGA R1** you may either use a USB-C cable, or the VIN pin.

If you're using the USB-C connector you must power it with 5V.

Powering the board with the VIN pin gives you more options, as you can safely power the board with any voltage between 6-24V.

By connecting the OFF pin to GND you can cut the power supply to the board, turning it off completely. Read more about this feature in the [OFF-pin](#) section of this article

It should however be noted that the internal operating voltage of the microcontroller is 3.3V, and you should not apply voltages higher than that to the GPIO pins.

## Installation

For detailed instructions on how to install the GIGA R1

 Board Package, please refer to the [Getting Started with GIGA R1](#) guide.

The **GIGA R1** can be programmed through:

The **Classic Arduino IDE 1.8.X**,  
the **Arduino IDE 2**,  
and the Web-editor.

## Board Package

The GIGA R1 is based on the [Arduino Mbed OS GIGA Board Package](#), which also provides a set of examples that works out of the box.

These examples are available in the Arduino IDE via **File > Examples > Examples for GIGA**.

### Mbed OS

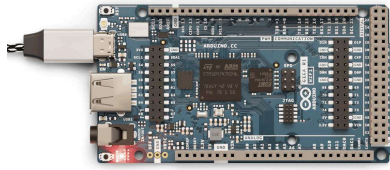
As the [Arduino Mbed OS GIGA Board Package](#) is based on [MbedOS](#), it is possible for the operating system to crash while running a sketch.

On most Arduino boards, when a sketch fails due to e.g. memory shortage, the board resets.

On the GIGA R1, whenever the MbedOS fails, the board does **not reset automatically**. Instead, if it fails, the onboard red LED will start to blink in a looping pattern of 4 fast blinks and 4 slow blinks.



Please note, the red LED does NOT mean your board is broken or bricked.



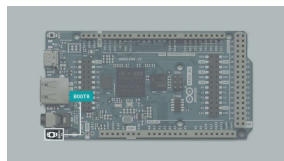
In case you encounter the red LED, you can either:

Press the reset button **once** (this resets the sketch).

**Double-tap** the reset button to enter bootloader mode (allowing you to re-program the board).

## Boot0

Pressing and holding the button labelled `BOOT0` on the board while powering the board will boot it into DFU-mode (Device Firmware Update), letting you reflash the bootloader without the use of external hardware, if you were to ever need to.



BOOT0 button

The state of this button can also be read from a sketch while it is running, giving you a basic interactive component without needing to do any wiring.

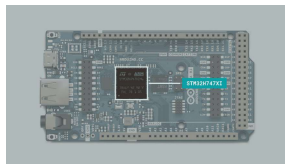
line of code:

```
1 digitalRead(PC_13);
```

## STM32H747XI Microcontroller

The GIGA R1 features the powerful dual core **STM32H747XI** microcontroller found on the Arduino PRO family's Portenta H7 board, but in a form factor accessible to any maker who has tinkered with an Arduino board before.

The **STM32H747XI** is a powerful dual core chip, capable of being programmed with a high-level language such as MicroPython on one core, while simultaneously running Arduino compiled code on the other, and having the two programs communicate with each other seamlessly.



Microcontroller on the  
GIGA R1

The microcontroller operates on a voltage of 3.3V, applying a higher voltage than that, such as 5V, to a pin might damage the microcontroller.

## RAM

The **GIGA R1** has 1 MB of SRAM that is internal to the processor, and 8MB of SDRAM which you can access and write to.

To access the SDRAM you need to use the SDRAM library, include it in your sketch with:

```
1 #include "SDRAM.h"
```

Before writing to the SDRAM, you need to allocate it, the following code will create an array that reserves 7MB of the SDRAM for you to write to.

```
1 uint8_t* myVeryBigArray
```

If you now store any data in this array, it will be written to SDRAM.

```
1 for (int i = 0; i<128;  
2     myVeryBigArray[  
3     }
```

When you no longer need to use the SDRAM, you can free it, so it can be used for other things.

```
1 SDRAM.free(myVeryBigArr
```

## Flash

The **GIGA R1** has 2MB of internal, and 16MB of external Flash storage. The external Flash storage on the **GIGA R1** is QSPI and can be accessed and used to store data. If you need to, you can configure the board to act as a USB flash drive, so you can store files such as images, audio, and more.

The GIGA firmware has full support for FATFS and littleFS.

To access the QSPI flash storage as a USB flash drive, you need to follow a few steps, first you need to update the WiFi modules firmware, then you need to create partitions on the flash storage, before finally exposing the partitions to be detected by a computer. These three steps are broken down into different built in example sketches that conveniently all come with the GIGA Board Package.

Firstly, navigate in the IDE menu to

```
File > Examples >  
STM32H747_System >  
WiFiFirmwareUpdater
```

and upload the sketch to your board.

In the Serial monitor you will now be able to interface with the

will now see the progress of the firmware update, don't power off the board until you see a message telling you that it is safe.

After completing this, the next step is to partition the flash storage. Navigate to

```
File > Examples >
STM32H747_System >
QSPIFormat
```

and upload the sketch. From here, your interaction with the board will be very similar to when you updated the WiFi firmware. In the serial monitor, you will get the option to choose from two partition schemes. For this purpose, partition scheme 1 is good. Again, send "y" in the serial monitor and wait for confirmation before powering the board off.

Lastly, navigate to

```
File > Examples > USB Mass
Storage >
AccessFlashAsUSBDisk
```

and upload the sketch.

Once this is completed, you should now see a new storage device connected as a portable storage device in your computer.

This can be very useful, as this flash storage **does not get deleted when you upload a new sketch to the board**. Meaning that you can store files on the board, and then upload a new sketch that can access and use those files without the need for an SD card and card reader.

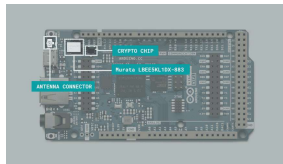
**\*Note: In this configuration, the USB serial port used for serial communication with the**



**information in the serial**

**monitor.** This includes uploading new sketches. To upload a new sketch you need to put the GIGA R1 in DFU mode by double pressing the RST button.\*\*\*

## Radio Module



Murata LBEE5KL1DX-883 radio module + antenna connector.

The Wi-Fi® / Bluetooth® module onboard the GIGA R1 WiFi is the Murata LBEE5KL1DX-883. This module does not come with a built-in antenna, but an external antenna is included when purchasing the board.

The antenna connector (see image above) is located right next to the USB-C connector, and is of a **U.FL** type.

### Wi-Fi®

Wi-Fi® on the GIGA R1 WiFi is supported via the [WiFi](#) library. This library is included in the core, so it is automatically installed when installing the Board Package.

To use the Wi-Fi® features on this board, please refer to the [GIGA R1 WiFi Network Examples](#) guide.

The easiest way to connect your board to the Internet is via the [Arduino Cloud](#)

 platform. Here you can configure, program, monitor and synchronize your devices without having to write any networking code.

## Bluetooth® Low Energy

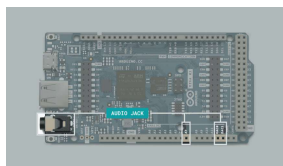
To use the BLE features on this board, refer to the [ArduinoBLE library documentation](#).

## Ethernet

If you want to add Ethernet connectivity to your project, the [Arduino Ethernet Shield Rev2](#) is compatible. In the shield's documentation, you will find a series of examples.

## Audio Jack

The **GIGA R1** features an audio jack, with 2x DAC channels, and 1x ADC channel, and is capable of reading input from a microphone, as well as outputting sound through a speaker.



Audio jack.

The audio jack is connected to the following pins:

## A13 / DAC1

### A7

Both of these come with caveats, though. As there is no amplifier circuit on the board itself, driving a high impedance speaker directly without an amplifier circuit could cause damage to the board, and microphone input without an amplifier circuit between the microphone and the board may sound dim.

In the coming sections we will provide resources and basic information on how to use the audio jack as both an input and an output.

## DAC Output

In order to output audio from the DAC, you can use the `AdvancedAnalogRedux` library from Arduino. You will need to connect a speaker to your board either through the audio jack or the DAC pins.

Include the library and create the `AdvancedDAC` object, such as the `dac1(A12)`, with the following code in the beginning of your sketch, outside of the `void setup()` function:

```
1 #include "AdvancedDAC.h"
2
3 AdvancedDAC dac1(A12);
```

To initialize the library, and check

```
1 if (!dac1.begin(AN_RES0
2     Serial.println(
3     while (1);
4 }
```

Then lastly, add the following code to `void loop()` to start:

```
1 if (dac1.available())
2
3     // Get a free
4     SampleBuffer b
5
6     // Write data
7     for (int i=0;
8         buf.data()
9     }
10
11     // Write the b
12     dac1.write(buf
13 }
```

**\*The options for audio playback and generation on your GIGA R1 are much more vast than this, however. To learn about audio playback in depth, check out the [GIGA R1 Audio Guide](#).\***

## ADC Input

The audio jack on the GIGA R1 is also connected to pin A7, for microphone capabilities.

To take advantage of this, you can use the `AdvancedAnalogRedux` library from Arduino, start off by including the library and setting up the pin as an `AdvancedADC`

```
1 #include "AdvancedADC.h"
2
3 AdvancedADC adc1(A7);
```

Now, initialize the library and run a check to make sure everything went as expected with the following code within

`void setup()`:

```
1 Serial.begin(9600);
2
3 // Resolution, sample
4 if (!adc1.begin(AN_R
5     Serial.println("
6     while (1);
7 }
```

Finally, read the ADC, and store it in a way that you can use it, do this within `void loop()`:

```
1 if (adc1.available()) {
2     SampleBuffer bu
3
4     // Print first
5     Serial.println(
6
7     // Release the
8     buf.release();
```

**\*The options for audio input on your GIGA R1 are much more vast than this, however. To learn about audio recording in depth, check out the [GIGA R1 Audio Guide](#).\***

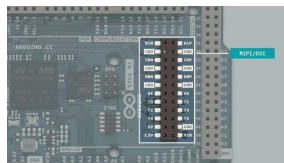
## MIPI DSI®

Display Serial Interface (DSI), is a specification from the Mobile Industry Processor Interface (MIPI).

The **STM32H747XI** has an internal 2D graphics accelerator with support for resolutions up to 1024x768, it also has the ability to encode and decode JPEG codec. This is what allows the **GIGA R1** to boast a 2 lane MIPI display interface.

The **GIGA Display Shield** is designed to be mounted on the GIGA R1 through the MIPI/DSI connector located on the board, with support for popular frameworks such as **LVGL** and **GFX**.

The pinout for the display connector is shown in the image below:



MIPI/DSI connector.

When connecting a module or shield to the GIGA R1

**i** WiFi board, be careful to not connect it at an angle or your board may be damaged.

The following pins are directly connected to the STM32H747XI and cannot be used as GPIOs.

D1P

CKN

CKP

D0N

D0P

The following pins can also be used as GPIOs:

D68

D69

D70

D71

D72

D73

D74

D75

The connector also has a series of power connections, including:

3.3 V

5 V

GND

VIN

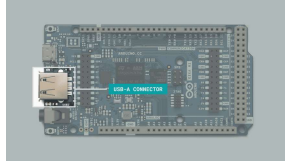
## USB Features

To learn about all the USB features in more detail, visit the [Guide to GIGA R1 USB Features](#).

### USB HID

The GIGA R1 comes with support for HID, and can be used as either a keyboard or mouse. For more information, visit the [USB HID section](#).

### USB Host



USB-A connector.

The USB-A port you find on the **GIGA R1** is configured as a host-only port, meaning that it cannot be used to program the board, instead it is used to connect peripherals to the board.

The board can receive keyboard input, effectively enabling a few hundred more inputs without any wiring, or be used to read & write files on a USB flash drive, which makes it possible to for example, build a [datalogger](#).

For more information, go to the following sections:

[USBHost Keyboard](#).

[USB mass storage](#).

## RTC

### RTC Manual Example

The following sketch will continuously print the time in the Serial monitor.





```
1  #include "mbed.h"
2  #include <mbed_mktime>
3
4  constexpr unsigned long
5  unsigned long printNow
6
7  void setup() {
8      Serial.begin(9600);
9      RTCset();
10 }
11
12 void loop() {
13     if (millis() >
14         Serial.print(
15         Serial.println(
16         printNow = mi
17     }
18 }
19
20 void RTCset() // Set
21 {
22     tm t;
23     t.tm_sec
24     t.tm_min
25     t.tm_hour
26     t.tm_mday
27     t.tm_mon
28     t.tm_year
```

To get accurate time, you'll want to change the values in `void RTCset()` to whatever time it is when you're starting this clock. As long as the VRTC pin is connected to power, the clock will keep ticking and time will be kept accurately.

## RTC / UDP / NTP Example

With the following sketch, you can automatically set the time by requesting the time from a Network Time Protocol (NTP), using the UDP protocol.

```
1  /*
2  Udp NTP Client
3
4  Get the time from a
5  Demonstrates use of
6  For more on NTP tim
7  see http://en.wikip
8
9  created 4 Sep 2010
10 by Michael Margolis
11 modified 9 Apr 2012
12 by Tom Igoe
13 modified 28 Dec 202
14 by Giampaolo Mancin
15
16 This code is in the
17 */
18
19 #include <WiFi.h>
20 #include <WiFiUdp.h>
21 #include <mbed_mktime.h>
22
23 int status = WL_IDLE
24 #include "arduino_se
25 //please enter
26 char ssid[] = ""; //
27 char pass[] = ""; //
28 int keyIndex = 0; //
```

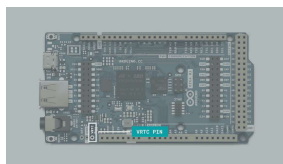
## RTC / UDP / NTP Example (Timezone)

This example provides an option to set the timezone. As the received epoch is based on GMT time, you can input e.g.  or  which represents the hours. The `timezone` variable is changed at the top of the example.

```
1  /*
2  Udp NTP Client with
3
4  Get the time from a
5  Demonstrates use of
6  For more on NTP tim
7  see http://en.wikip
8
9  created 4 Sep 2010
10 by Michael Margolis
11 modified 9 Apr 2012
12 by Tom Igoe
13 modified 28 Dec 202
14 by Giampaolo Mancin
15 modified 29 Jan 202
16 by Karl Söderby
17
18 This code is in the
19 */
20
21 #include <WiFi.h>
22 #include <WiFiUdp.h>
23 #include <mbed_mktime
24
25 int timezone = -1; /
26
27 int status = WL_IDLE
28
```

## VRTC Pin

The GIGA R1 also features a VRTC pin, giving you the ability to power the RTC with a coin cell battery to keep the time even when your board is turned off, for low power timekeeping.



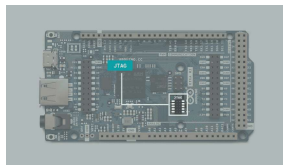
VRTC pin on the GIGA  
R1

The Arduino GIGA features an onboard Arducam compatible connector.

To learn more about the camera capabilities of the GIGA R1, check out the [GIGA R1 Camera Guide](#)

## JTAG

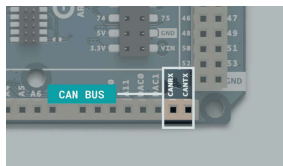
The **GIGA R1** features a 2x5 pin JTAG (Joint Test Action Group) connector, giving advanced debug functionalities for more advanced users.



JTAG Connector

## CAN Bus

The **GIGA R1** features a dedicated CAN bus.



CAN Bus

The CAN bus does not include a built in transceiver. If you need to use the CAN bus, you can add a transceiver as a breakout board.



CAN, or **Controller Area**

microcontroller-based devices to communicate with each other without the need for a host computer. This means that building a complex system with many different subsystems within becomes much easier.

This makes the **GIGA R1** a powerful option for complex multilayered systems, as it can be integrated into existing systems or be used to build a new one from scratch.

The CAN pins on the **GIGA R1** are labelled `CANRX` and `CANTX`. Typically, transceiver breakouts are labelled with a similar syntax, and to connect them to the board, use the following wiring scheme:

| GIGA R1 | Transceiver |
|---------|-------------|
| VCC     | 3.3V        |
| GND     | GND         |
| CANTX   | CANTX*      |
| CANRX   | CANRX*      |

 \*The name of CANTX/CANRX differs from product to product.

Below is an example of how to send & receive data using a CAN bus.

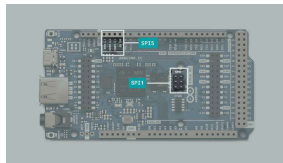


```

1  #include "CAN.h"
2
3  mbed::CAN can1(PB_5,
4  uint8_t counter = 0;
5
6  void send() {
7      Serial.println("sen
8      if (can1.write(mbed
9          Serial.println("w
10         counter++;
11         Serial.print("Mes
12         Serial.println(co
13     } else {
14         Serial.println("T
15         Serial.println(ca
16         can1.reset();
17     }
18 }
19
20 void receive() {
21     mbed::CANMessage ms
22     if (can1.read(msg))
23         Serial.print("Mes
24         Serial.println(ms
25     }
26 }
27
28

```

## SPI



SPI Pins

The **GIGA R1** features two separate SPI (Serial Peripheral Interface) buses, one is configured on the 6 pin header (ICSP) labelled SPI, and the other is broken out into pin connections on the board.

The first bus which has a dedicated SPI header `SPI1` uses

(CIPO) - D89

(COPI) - D90

(SCK) - D91

(CS) - unassigned, use any free GPIO for this.

Please note that the SPI header provides a 5 V pin.

Make sure that the SPI device you are connecting

 supports an input voltage of 5 V. If you have an SPI device that supports 3.3 V only, use the `SPI5` port (see below).

The second bus (header), `SPI5`, uses the following pins:

(CIPO) - D12

(COPI) - D11

(SCK) - D13

(CS) - D10

For using both SPI buses simultaneously, check out the following example:

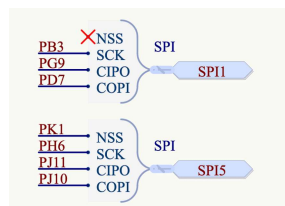


```

1  #include <SPI.h>
2
3  const int CS_1 = 10;
4  const int CS_2 = 5;
5
6  void setup() {
7      pinMode(CS_1, OUTPUT);
8      pinMode(CS_2, OUTPUT);
9
10     SPI.begin();
11     SPI1.begin();
12
13     digitalWrite(CS_1, LOW);
14     digitalWrite(CS_2, LOW);
15
16     SPI.transfer(0x00);
17     SPI1.transfer(0x00);
18
19     digitalWrite(CS_1, HIGH);
20     digitalWrite(CS_2, HIGH);
21 }
22
23 void loop() {
24 }

```

Please note that the in the GIGA R1 schematics and the code does not match exactly. In the schematics, you will notice that `SPI` is `SPI1` and `SPI1` is `SPI5`.



SPI ports in the schematics.

## I2C Pins

I2C lets you connect multiple I2C compatible devices in series using



I2C bus to a 7 bit address, meaning that the technical limit of I2C devices on a single line is 128. Practically, you're never gonna reach 128 devices before other limitations kick in.

The **GIGA R1** has three separate I2C buses of which two are usable without external components, letting you control more devices.

The pins used for I2C on the **GIGA R1** are the following:

SDA - D20

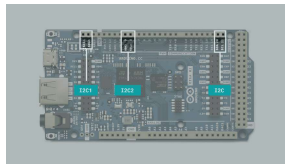
SCL - D21

SDA1 - also available on the camera connector.

SCL1 - also available on the camera connector.

SDA2 - D9

SCL2 - D8



I2C Pins

To connect I2C devices you will need to include the [Wire](#) library at the top of your sketch.

```
1 #include <Wire.h>
```

Inside `void setup()` you need to initialize the library, and initialize the I2C port you want to use.

```
1 Wire.begin() //SDA & SD
2 Wire1.begin(); //SDA1 &
3 Wire2.begin(); //SDA2 &
```

And to write something to a device connected via I2C, we can use the following commands:

```
1 Wire.beginTransmission(
2 Wire.write(byte(0x00));
3 Wire.write(val); //send
4 Wire.endTransmission();
```

If you pay close attention you may notice that there are three sets of I2C pins. The two first sets (SDA, SCL, SDA1, SCL1) have internal pullup resistors connected to them which are required to make them function as I2C pins.

If you want to use the third set (SDA2, SCL2) as I2C pins you will need to use external pull-up resistors.

## Serial/UART Pins

The **GIGA R1** supports, like every other Arduino board, serial communication with UART (Universal Asynchronous, Receiver-Transmitter). However, the **GIGA R1** board features 5 separate serial ports, including the standard serial over USB port that is initialized using `Serial.begin()`.

This not only means that you may

ports and monitor them separately, which is useful enough in and of itself, but that you may also communicate with **4 different serial enabled devices** simultaneously.

The pins used for UART on the **GIGA R1** are the following:

RX0 - D0  
TX0 - D1  
RX1 - D19  
TX1 - D18  
RX2 - 17  
TX2 - 16  
RX3 - 15  
TX3 - 14

Each Serial port works in the same way as the one you're used to, but you use different functions to target them:

```
1 Serial.begin(9600); //i
2 Serial1.begin(9600); //
3 Serial2.begin(9600); //
4 Serial3.begin(9600); //
5 Serial4.begin(9600); //
```

To send and receive data through UART, we will first need to set the baud rate inside `void setup()`. In this example, we use RX0/TX0.

```
1 Serial1.begin(9600);
```

To read incoming data, we can use a while loop() to read each individual character and add it to

```
1 while(Serial1.available
2     delay(2);
3     char c = Serial1.re
4     incoming += c;
5 }
```

And to write something, we can use the following command:

```
1 Serial1.write("Hello wo
```

## Pins

The **GIGA R1** gives you access to more pins than any other Arduino board that is this accessible for makers. Many of them have special features that will be accounted for in the upcoming sections of this article. Keep reading to learn what you can do with them.

If you just need a quick overview of the pins functionality, this is a full table of all the IO pins on the **GIGA R1**

| Pin | Function | Notes                |
|-----|----------|----------------------|
| D0  | TX       | Serial communication |
| D1  | RX       | Serial communication |
| D2  | PWM      | PWM, Digital IO pin  |
|     |          | PWM, Digital IO      |

| Pin | Function | Notes                |
|-----|----------|----------------------|
| D4  | PWM      | PWM, Digital IO pin  |
| D5  | PWM      | PWM, Digital IO pin  |
| D6  | PWM      | PWM, Digital IO pin  |
| D7  | PWM      | PWM, Digital IO pin  |
| D8  | PWM/SCL2 | PWM, Digital IO, I2C |
| D9  | PWM/SDA2 | PWM, Digital IO, I2C |
| D10 | PWM/CS   | PWM, Digital IO, SPI |
| D11 | PWM/COPI | PWM, Digital IO, SPI |
| D12 | PWM/CIPO | PWM, Digital IO, SPI |
| D13 | PWM/SCK  | PWM, Digital IO, SPI |
| D14 | TX3      | Serial communication |
| D15 | RX3      | Serial communication |
| D16 | TX2      | Serial communication |
| D17 | RX2      | Serial communication |
| D18 | TX1      | Serial communication |
| D19 | RX1      | Serial communication |
| D20 | SDA      | Digital IO, I2C      |
| D21 | SCL      | Digital IO, I2C      |
| D22 | GPIO     | Digital IO pin       |
| D23 | GPIO     | Digital IO pin       |

| Pin | Function | Notes          |
|-----|----------|----------------|
| D25 | GPIO     | Digital IO pin |
| D26 | GPIO     | Digital IO pin |
| D27 | GPIO     | Digital IO pin |
| D28 | GPIO     | Digital IO pin |
| D29 | GPIO     | Digital IO pin |
| D30 | GPIO     | Digital IO pin |
| D31 | GPIO     | Digital IO pin |
| D32 | GPIO     | Digital IO pin |
| D33 | GPIO     | Digital IO pin |
| D34 | GPIO     | Digital IO pin |
| D35 | GPIO     | Digital IO pin |
| D36 | GPIO     | Digital IO pin |
| D37 | GPIO     | Digital IO pin |
| D38 | GPIO     | Digital IO pin |
| D39 | GPIO     | Digital IO pin |
| D40 | GPIO     | Digital IO pin |
| D41 | GPIO     | Digital IO pin |
| D42 | GPIO     | Digital IO pin |
| D43 | GPIO     | Digital IO pin |
| D44 | GPIO     | Digital IO pin |
| D45 | GPIO     | Digital IO pin |
| D46 | GPIO     | Digital IO pin |
| D47 | GPIO     | Digital IO pin |
| D48 | GPIO     | Digital IO pin |
| D49 | GPIO     | Digital IO pin |
| D50 | GPIO     | Digital IO pin |
| D51 | GPIO     | Digital IO pin |
| D52 | GPIO     | Digital IO pin |
| D53 | GPIO     | Digital IO pin |
| D54 | GPIO     | Digital IO pin |
| D55 | GPIO     | Digital IO pin |
| D56 | GPIO     | Digital IO pin |

| Pin         | Function  | Notes          |
|-------------|-----------|----------------|
| D58         | GPIO      | Digital IO pin |
| D59         | GPIO      | Digital IO pin |
| D60         | GPIO      | Digital IO pin |
| D61         | GPIO      | Digital IO pin |
| D62         | GPIO      | Digital IO pin |
| D63         | GPIO      | Digital IO pin |
| D64         | GPIO      | Digital IO pin |
| D65         | GPIO      | Digital IO pin |
| D66         | GPIO      | Digital IO pin |
| D67         | GPIO      | Digital IO pin |
| D68         | GPIO      | Digital IO pin |
| D69         | GPIO      | Digital IO pin |
| D70         | GPIO      | Digital IO pin |
| D71         | GPIO      | Digital IO pin |
| D72         | GPIO      | Digital IO pin |
| D73         | GPIO      | Digital IO pin |
| D74         | GPIO      | Digital IO pin |
| D75         | GPIO      | Digital IO pin |
| A0 /<br>D76 | Analog in | Analog In      |
| A1 /<br>D77 | Analog in | Analog In      |
| A2 /<br>D78 | Analog in | Analog In      |
| A3 /<br>D79 | Analog in | Analog In      |
| A4 /<br>D80 | Analog in | Analog In      |
| A5 /<br>D81 | Analog in | Analog In      |
| A6 /<br>D82 | Analog in | Analog In      |
| A7 /<br>D83 | Analog in | Analog In      |

| Pin             | Function        | Notes                                   |
|-----------------|-----------------|-----------------------------------------|
| A9              | Analog in       | Analog In                               |
| A10             | Analog in       | Analog In                               |
| A11             | Analog in       | Analog In                               |
| A12<br>/<br>D84 | DAC0            | Analog In, DAC                          |
| A13<br>/<br>D85 | DAC1            | Analog In, DAC                          |
| D86             | RGB (red)       | Only RGB, not accessible as GPIO        |
| D87             | RGB (green)     | Only RGB, not accessible as GPIO        |
| D88             | RGB (blue)      | Only RGB, not accessible as GPIO        |
| D89             | SPI1 (CIPO)     | SPI connector                           |
| D90             | SPI1 (COPI)     | SPI connector                           |
| D91             | SPI1 (SCK)      | SPI connector                           |
| D92             | USB Host Enable | USB-A connector, not accessible as GPIO |
| D93             | CANRX           | Digital IO pin, CAN                     |
| D94             | CANTX           | Digital IO pin, CAN                     |

### Analog Pins

The **GIGA R1** has 12 analog input pins that can be read with a resolution of 16 Bits, by using the `analogRead()` function.



```
1 value = analogRead(pin,
```

The reference voltage of these pins is 3.3V.

Pins A8, A9, A10 and A11 can not be used as GPIOs, but are limited to use as analog input pins.

The **STM32H7** has an internal OPAMP and comparator that are exposed on the **GIGA R1** as follows:

| Pin | OPAMP                 | Comparator |
|-----|-----------------------|------------|
| A0  | OPAMP1_VOUT           | COMP1_INM  |
| A1  | OPAMP1_VINM<br>&VINM0 |            |
| A2  | OPAMP1_VINP           | COMP1_INP  |
| A3  |                       | COMP1_INM  |
| A6  |                       | COMP1_INM  |

For more advanced analog readings, you can use the **AdvancedAnalogRedux** library. Read more about this in the [Advanced ADC section](#).

### PWM Pins

PWM (Pulse Width Modulation) capability allows a digital pin to emulate analog output by flickering on and off very fast letting you, among other things, dim LEDs connected to digital pins.

The **GIGA R1** has 12 PWM capable pins, the PWM capable pins are 2-

```
1 analogWrite(pin, value)
```

## Digital Pins

The **GIGA R1** features more pins than any other Arduino board for makers, a full 76 digital pins. Though many of them serve another purpose and shouldn't be used for GPIO if you have other pins available.

0 - RX0

1 - TX0

8 - SCL2

9 - SDA2

10 - CS

11 - COPI

12 - CIPO

13 - SCK

14 - TX3

15 - RX3

16 - TX2

17 - RX2

18 - TX1

19 - RX1

20 - SDA

21 - SCL

The reference voltage of all digital pins is 3.3V.

The logic for `LED_BUILTIN` is reversed if compared to the behavior of, for example, the **Arduino UNO** board. What this means is that if you write HIGH to `LED_BUILTIN`, the LED will turn off, and on respectively if you

## D7 Pin

By default, the digital pin 7 (D7) provides a voltage of ~1.65 V.

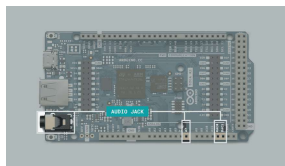
To disable this pin, you need to configure it as an output and set it to a `LOW` state.

```
1 pinMode(7, OUTPUT);  
2 digitalWrite(7, LOW);
```

## DAC Pins

The **GIGA R1** also has two DAC pins, A12 & A13, that can act as genuine analog output pins which means they are even more capable than PWM pins.

```
1 analogWrite(pin, value)
```



DAC Pins

These DAC pins have a default write resolution of 8-bits. This means that values that are written to the pin should be between 0-255.

However you may change this write resolution if you need to, to up to 12-bits:

```
1 analogWriteResolution(1023);
```

For advanced usage of the DAC, you can use the



`AdvancedAnalogRedux`

library. Read more about this in the [Advanced DAC section](#).

## OFF Pin

On the **GIGA R1** you will find a pin labelled "**OFF**". If you connect this pin to ground, the board will power down even if power is supplied to the board.

You can install a flip-switch to the board to let you turn your device on and off easily, which can be a handy option for a more permanent fixture.

## Interrupts

If you're creating a project that relies heavily on accurate sensor data, and therefore need to ensure that you read the record any change in value, it can be difficult to write a program that does anything else well. This is because the microcontroller is busy trying to read the values constantly. To get around this you can use interrupts that can let you can be useful for reading input from for example a rotary encoder or a push button without putting any code in your loop function

This feature might be extra valuable to the maker with an **GIGA R1**, as their circuit gets more and more complex.

All GPIO pins on the **GIGA R1** can be used for interrupts.

The syntax for creating an interrupt function should be included in `void setup()` and is as follows:

```
1 attachInterrupt(digital
```

`pin` represents the pin number of the pin your input sensor is connected to,

`ISR` is the function that is called whenever the interrupt is triggered, and should be defined by you somewhere in your sketch.

`mode` defines when the interrupt should be triggered, and can be one of four pre-defined modes.

The different modes that can be used are:

`LOW` triggers the interrupt when the pin is low.

`CHANGE` triggers whenever the pin changes values.

`RISING` triggers when the pin goes from low to high.

`FALLING` triggers when the pin goes from high to low.

This example sketch will turn on or off an LED connected to pin 13 whenever a pushbutton connected to pin 2 is pressed or

```
1  const byte ledPin = 13;
2  const byte interruptPin = 2;
3  volatile byte state = 0;
4
5  void setup() {
6    pinMode(ledPin, OUTPUT);
7    pinMode(interruptPin, INPUT);
8    attachInterrupt(digitalPinToInterrupt(interruptPin), blink, RISING);
9  }
10
11 void loop() {
12   digitalWrite(ledPin, state);
13 }
14
15 void blink() {
16   state = !state;
17 }
```

| Suggest changes                                                                                                                                                                                  | Need support?                                                                                                                       | License                                                                                                               |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| The content on <a href="https://docs.arduino.cc">docs.arduino.cc</a> is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page <a href="#">here</a> . | <a href="#">Help Center</a><br><a href="#">Ask the Arduino Forum</a><br><a href="#">Discover Arduino</a><br><a href="#">Discord</a> | The Arduino documentation is licensed under the <a href="#">Creative Commons Attribution-Share Alike 4.0</a> license. |

Was this article helpful?



